

Probabilistic Graphical Models

CSE 5830

Inference

Lecture 8

Variable Elimination

Goals for Today

- Exact inference
- Variable elimination
- Variable elimination algorithm
- Complexity of variable elimination
- Elimination as graph operation

Probabilistic Inference

- Assume we are given a graphical model (such as a Bayes net or an MRF), we are interested in using it to answer useful questions.
 - Marginal inference
 - what is the probability of a given variable in our model after we sum everything else out?

$$P(\mathbf{X} \mid \mathbf{E} = \mathbf{e}) = \frac{P(\mathbf{X}, \mathbf{E} = \mathbf{e})}{P(\mathbf{E} = \mathbf{e})}$$

X: Query variable

E: Evidence variable

Y: other variables

$$\propto P(\mathbf{X}, \mathbf{E} = \mathbf{e})$$

$$= \sum_{\mathbf{y} \in \text{Val}(\mathbf{Y})} P(\mathbf{X}, \mathbf{E} = \mathbf{e}, \mathbf{Y} = \mathbf{y})$$

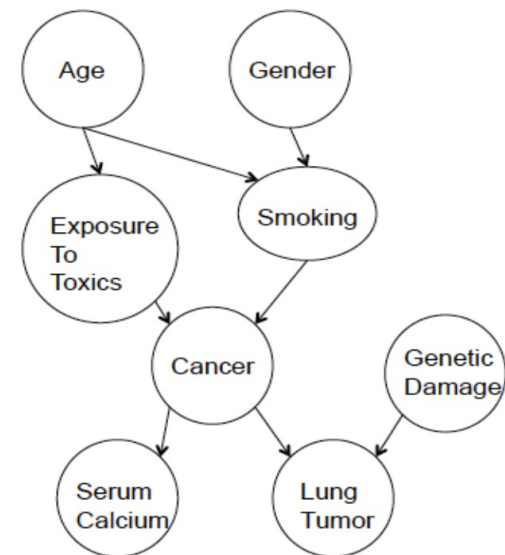
Example

What is the probability of having cancer given there is calcium in the blood?

- $\chi = \{A, Ge, E, Sm, C, Gd, Se, L\}$
 - Evidence variable is Se (serum Calcium)
 - Query variable is C cancer

$$P(C | Se) = \frac{P(C, Se)}{P(Se)}$$

$$\begin{aligned}
 P(C, Se) &= \sum_{A, Ge, E, Sm, L, Gd} P(A, Ge, E, Sm, Se, C, L, Gd) \\
 &= \sum_{A, Ge, E, Sm, L, Gd} P(A)P(Ge)P(Gd)P(E | A), P(Sm | A, G)P(Se | C)P(C | E, Sm)P(L | C, Gd) \\
 P(Se) &= \sum_C P(C, Se)
 \end{aligned}$$

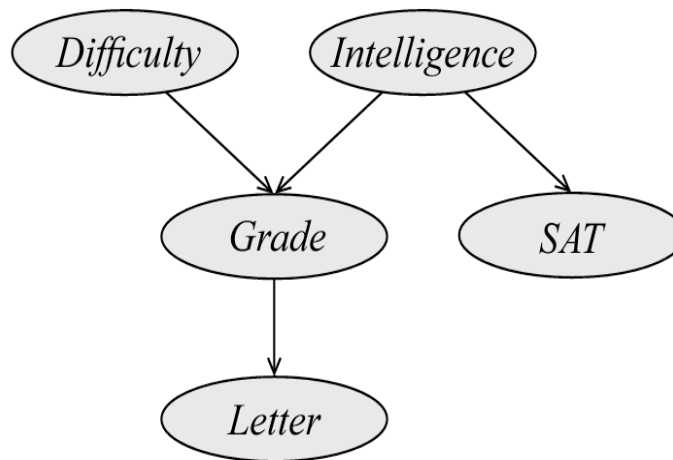


Example

- What is the probability of having a good recommendation letter given the SAT score is high?

Joint distribution $P(D, I, S, G, L)$

$P(L|S)$



Probabilistic Inference

- In general it is intractable
- In practice we can often do it anyway
 - *Exact inference*
 - via variable elimination
 - *Approximate inference*
 - *Maximum a posteriori inference*
 - find the best explanation, not necessarily the whole distribution

Exact Inference

- $P(D)$?

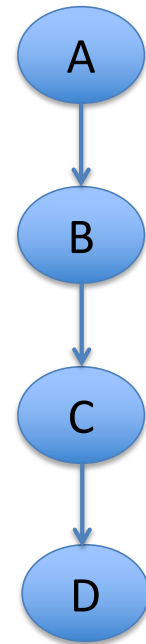
$$\begin{aligned} P(D) &= \sum_{A,B,C} P(A, B, C, D) \\ &= \sum_{A,B,C} P(A)P(B|A)P(C|B)P(D|C) \end{aligned}$$

- First, Let's calculate $P(B)$ from things we have.

$$P(B) = \sum_{a \in \text{Val}(A)} P(A = a)P(B | A = a)$$

What is the complexity of calculating $P(B)$ assuming A has k and B has m values?

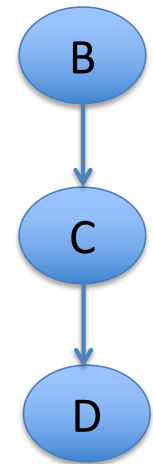
$$O(k \times m)$$



Elimination in chains

- We now have a Bayesian network for the marginal distribution $P(B, C, D)$.
- We can repeat the same process to calculate $P(C)$.

$$P(C) = \sum_{b \in \text{Val}(B)} P(B = b)P(C \mid B = b)$$



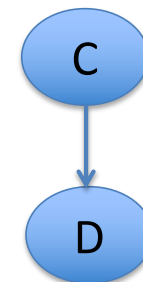
If B and C have k values each, complexity is $O(k^2)$

Elimination in chains

- We now have $P(C, D)$

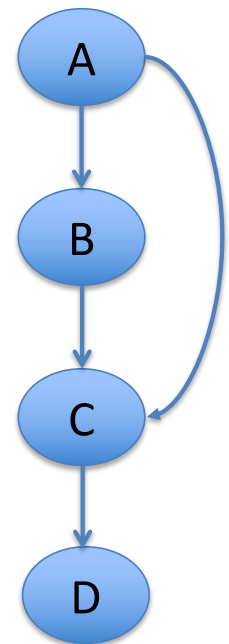
$$P(D) = \sum_{c \in \text{Val}(C)} P(C = c)P(D \mid C = c)$$

- Marginalizing out A and B happened in two steps
- Exploiting the Bayesian network structure.



Computation depends on Structure

- Structure of BN is critical for computation
 - If A had been a parent of C,
 - $P(C) = \sum_b P(b)P(C | b)$ wouldn't have sufficed
- Algorithm does not compute single values but sets of values at a time
 - $P(B)$ over all possible values of B are used to compute $P(C)$



Elimination in chains reduces the complexity

- n variables in chain $X_1 \rightarrow \dots \rightarrow X_n$
- Each variable has k values
 - What is the complexity of computing $P(X_1, \dots, X_n)$?
- Compute $P(X_{i+1})$ from $P(X_i)$

$$P(X_{i+1}) = \sum_{x_i} P(X_{i+1} \mid x_i) P(x_i)$$

- Cost of each step is $O(k^2)$
- Total cost $O(nk^2)$
- Generating the joint distribution: k^n

For large k and $n \rightarrow nk^2 < k^n$

Elimination in chains

Notice that the same step happened for each random variable:

- We created a new CPD over a variable and its “successor”
- We summed out (marginalized) the variable.

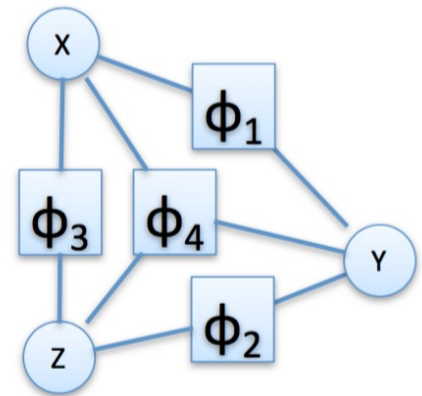
$$\begin{aligned} P(D) &= \sum_{a \in \text{Val}(A)} \sum_{b \in \text{Val}(B)} \sum_{c \in \text{Val}(C)} P(A = a)P(B = b \mid A = a)P(C = c \mid B = b)P(D \mid C = c) \\ &= \sum_{c \in \text{Val}(C)} P(D \mid C = c) \sum_{b \in \text{Val}(B)} P(C = c \mid B = b) \sum_{a \in \text{Val}(A)} P(A = a)P(B = b \mid A = a) \end{aligned}$$

Exploiting network structure for elimination in chains

- Not explicitly obtain the joint distribution
- Exponential blowup avoided!
 - We reused computation from previous steps and avoided doing the same work more than once.
 - Dynamic programming!
 - We exploited the Bayesian network structure
 - each sub expression only depends on a small number of variables.

Variable elimination

- But is there a general technique for any graphical model, not just chains?
 - We can use factor graphs to define an algorithm for BN and MM
- Factor Graphs
 - Bipartite graph
 - Variable nodes (circles)
 - Factor nodes (squares)
 - Edge between variable and factor if the factor depends on that variable.
 - Makes the factors more obvious.
 - CPDs can be seen as factors.



Variable elimination

Sum-product variable elimination

- The scope of the factors is limited
- Product of only a subset of factors

$$P(A, B, C, D) = \phi_A \cdot \phi_B \cdot \phi_C \cdot \phi_D$$

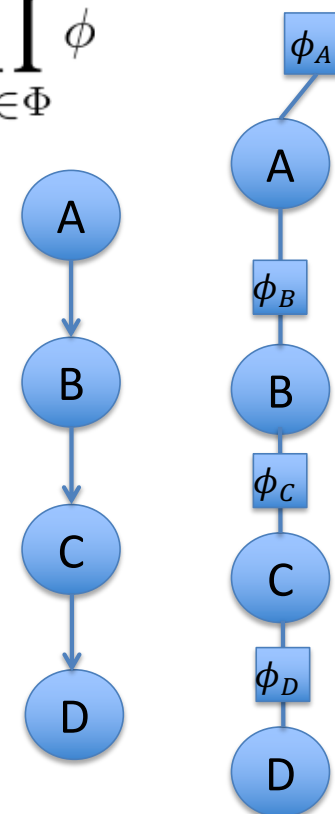
$$P(D) = \sum_C \sum_B \sum_A P(A, B, C, D).$$

$$\begin{aligned} P(D) &= \sum_C \sum_B \sum_A \phi_A \cdot \phi_B \cdot \phi_C \cdot \phi_D \\ &= \sum_C \sum_B \phi_C \cdot \phi_D \cdot \left(\sum_A \phi_A \cdot \phi_B \right) \\ &= \sum_C \phi_D \cdot \left(\sum_B \phi_C \cdot \left(\sum_A \phi_A \cdot \phi_B \right) \right) \end{aligned}$$

$$\sum_Z \prod_{\phi \in \Phi} \phi$$

Two main operation:

- Marginalization
- Factor product

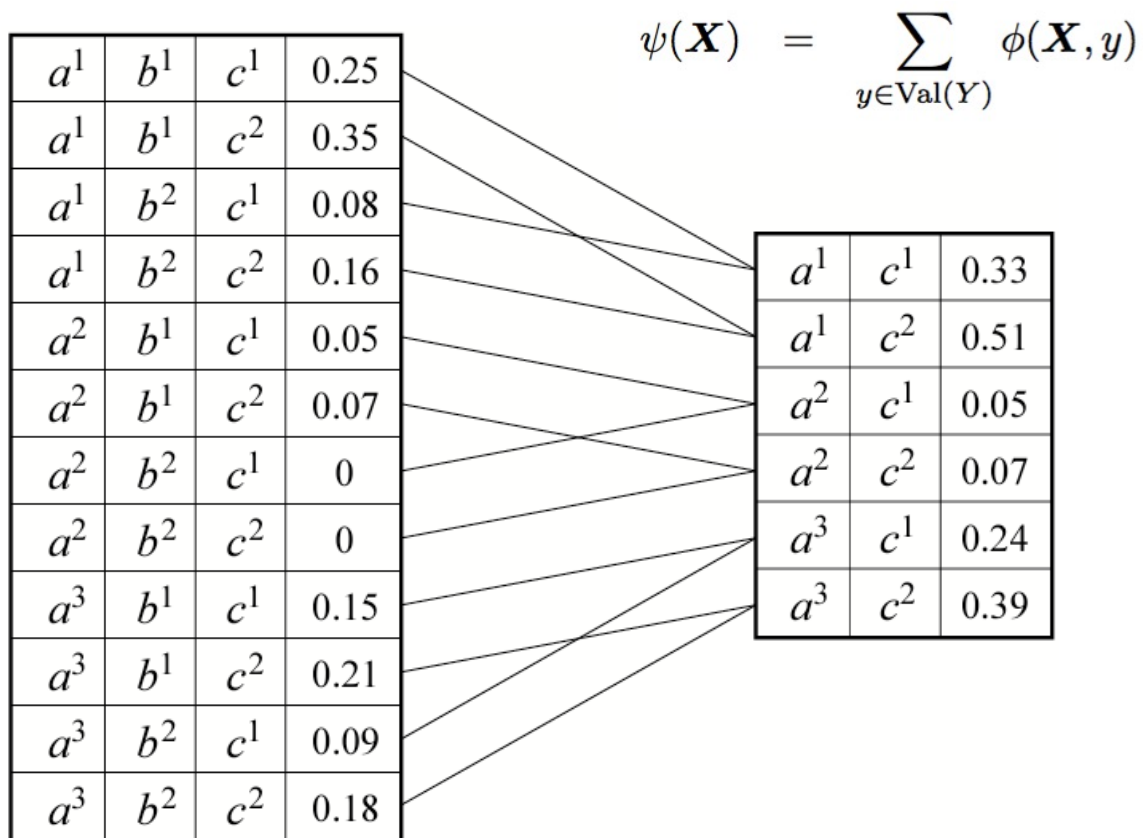


Factor marginalization

- The key operation when computing probabilities is marginalization
- Given $\mathbf{X}$ and Y ($Y \notin \mathbf{X}$), we can turn a factor $\phi(\mathbf{X}, Y)$ into a factor $\psi(\mathbf{X})$ via marginalization:

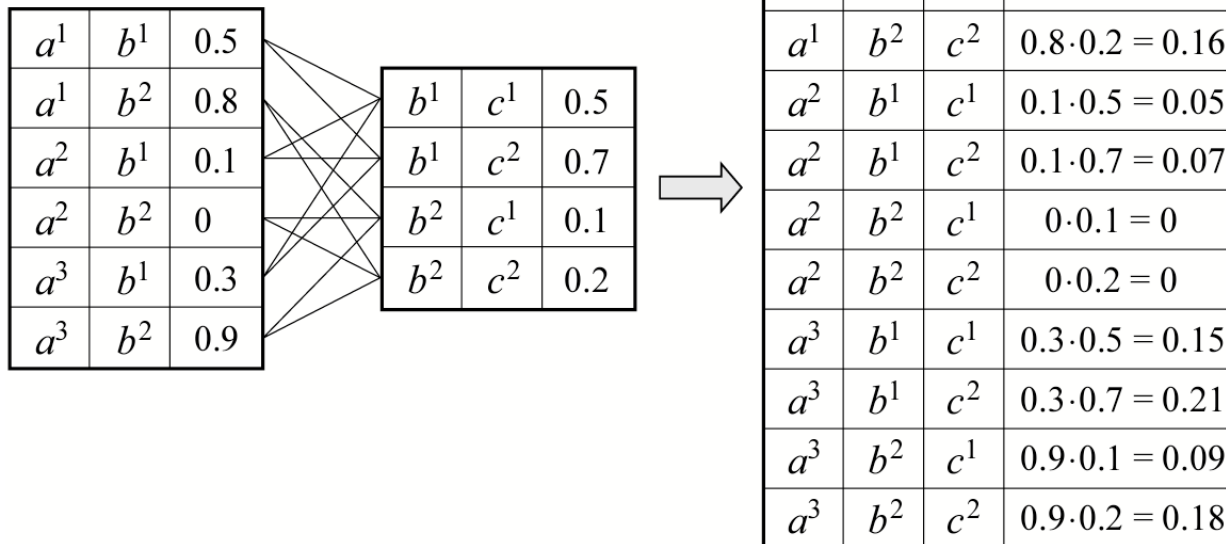
$$\psi(\mathbf{X}) = \sum_{y \in \text{Val}(Y)} \phi(\mathbf{X}, y)$$

Revisited: Factor marginalization



Factor product

$$\psi_k(\mathbf{X}_k) = \prod_{i=1}^{m_k} \phi_i$$



Factors are like numbers

- Products are commutative: $\phi_1 \cdot \phi_2 = \phi_2 \cdot \phi_1$
- Products are associative:
$$(\phi_1 \cdot \phi_2) \cdot \phi_3 = \phi_1 \cdot (\phi_2 \cdot \phi_3)$$
- Sums are commutative: $\sum_X \sum_Y \phi = \sum_Y \sum_X \phi$
- Distributivity of multiplication over summation:

$$X \notin \text{Scope}(\phi_1) \Rightarrow \sum_X (\phi_1 \cdot \phi_2) = \phi_1 \cdot \sum_X \phi_2$$

Algorithm

Eliminating One Variable

Input: Set of factors Φ , variable Z to eliminate

Output: new set of factors Ψ

1. Let $\Phi' = \{\phi \in \Phi \mid Z \in \text{Scope}(\phi)\}$
2. Let $\Psi = \{\phi \in \Phi \mid Z \notin \text{Scope}(\phi)\}$
3. Let ψ be $\sum_Z \prod_{\phi \in \Phi'} \phi$
4. Return $\Psi \cup \{\psi\}$

Algorithm

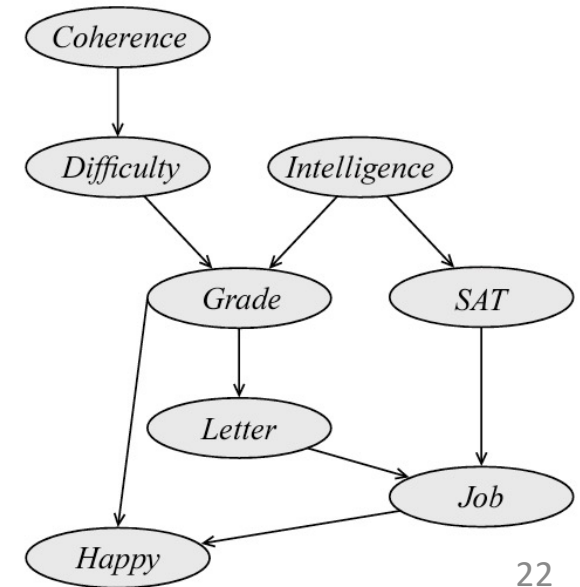
Variable Elimination

- Input: Set of factors Φ , ordered list of variables Z to eliminate
 - Output: new factor ψ
1. For each $Z_i \in Z$ (in order):
 - Let $\Phi = \text{Eliminate} - \text{One}(\Phi, Z_i)$
 2. Return $\prod_{\phi \in \Phi} \phi$

Variable Elimination

- Example: $P(J)$
 - Write down in a factor forms, CPDs can be consider as factors
 - Marginalize
 - In general, any marginal prob. Involves taking product of all the CPDs and doing a summation on all the variables except the query one

$$\begin{aligned} & P(C, D, I, G, S, L, J, H) \\ = & P(C)P(D | C)P(I)P(G | I, D)P(S | I) \\ & P(L | G)P(J | L, S)P(H | G, J) \\ = & \phi_C(C)\phi_D(D, C)\phi_I(I)\phi_G(G, I, D)\phi_S(S, I) \\ & \phi_L(L, G)\phi_J(J, L, S)\phi_H(H, G, J). \end{aligned}$$



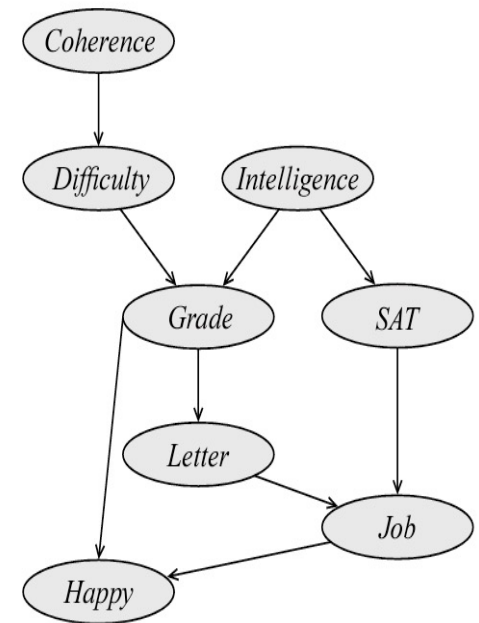
Variable Elimination

- $P(J)$
- Eliminate C, D, I, H, G, S, L

Eliminate variables one at a time.

1. multiply all the factors that mention that variable
2. generate a product factor
3. sum out $\rightarrow$ marginalization

$$\sum_{L, S, G, H, I, D, C} \phi_J(J, L, S) \phi_L(L, G) \phi_S(S, I) \phi_G(G, I, D) \phi_H(H, G, J) \phi_I(I) \phi_D(C, D) \phi_C(C)$$



Variable Elimination

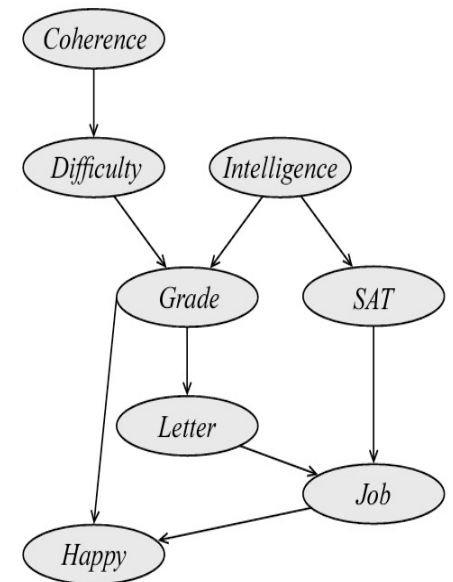
- $P(J)$, order C, D, I, H, G, S, L
- Eliminate C

$$\sum_{L, S, G, H, I, D, C} \phi_J(J, L, S) \phi_L(L, G) \phi_S(S, I) \phi_G(G, I, D) \phi_H(H, G, J) \phi_I(I) \phi_D(C, D) \phi_C(C)$$

$$\sum_{L, S, G, H, I, D} \phi_J(J, L, S) \phi_L(L, G) \phi_S(S, I) \phi_G(G, I, D) \phi_H(H, G, J) \phi_I(I) \sum_C \phi_D(C, D) \phi_C(C)$$

$$\psi_1(D) = \sum_C \phi_D(C, D) \phi_C(C)$$

$$\sum_{L, S, G, H, I, D} \phi_J(J, L, S) \phi_L(L, G) \phi_S(S, I) \phi_G(G, I, D) \phi_H(H, G, J) \phi_I(I) \psi_1(D)$$



Variable Elimination

- $P(J)$, order C,D,I,H,G,S,L

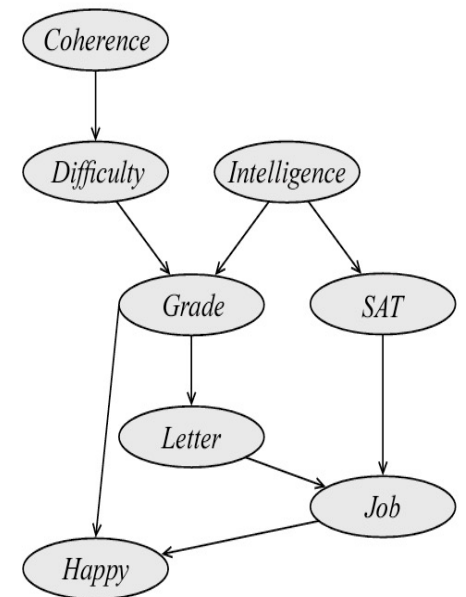
$$\sum_{L,S,G,H,I,D} \phi_J(J,L,S) \phi_L(L,G) \phi_S(S,I) \phi_G(G,I,D) \phi_H(H,G,J) \phi_I(I) \psi_1(D)$$

- Eliminate D

$$\sum_{L,S,G,H,I} \phi_J(J,L,S) \phi_L(L,G) \phi_S(S,I) \phi_H(H,G,J) \phi_I(I) \sum_D \phi_G(G,I,D) \psi_1(D)$$

$$\psi_2(G,I) = \sum_D \phi_G(G,I,D) \psi_1(D)$$

$$\sum_{L,S,G,H,I} \phi_J(J,L,S) \phi_L(L,G) \phi_S(S,I) \phi_H(H,G,J) \phi_I(I) \psi_2(G,I)$$



Variable Elimination

- $P(J), C, D, I, H, G, S, L$

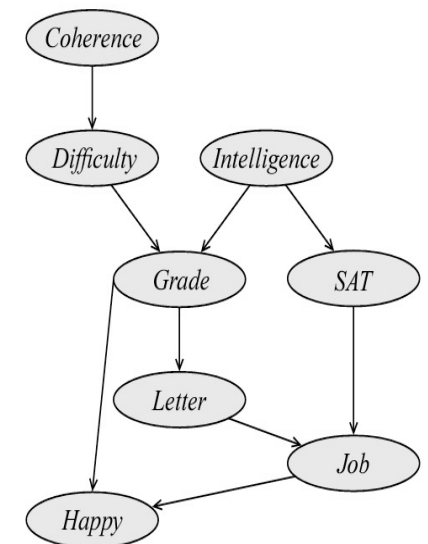
$$\sum_{L, S, G, H, I} \phi_J(J, L, S) \phi_L(L, G) \phi_S(S, I) \phi_H(H, G, J) \phi_I(I) \psi_2(G, I)$$

- Eliminate I

$$\sum_{L, S, G, H} \phi_J(J, L, S) \phi_L(L, G) \phi_H(H, G, J) \sum_I \phi_S(S, I) \phi_I(I) \psi_2(G, I)$$

$$\psi_3(S, G) = \sum_I \phi_S(S, I) \phi_I(I) \psi_2(G, I)$$

$$\sum_{L, S, G, H} \phi_J(J, L, S) \phi_L(L, G) \phi_H(H, G, J) \psi_3(S, G)$$



Variable Elimination

- $P(J)$, order C,D,I,H,G,S,L

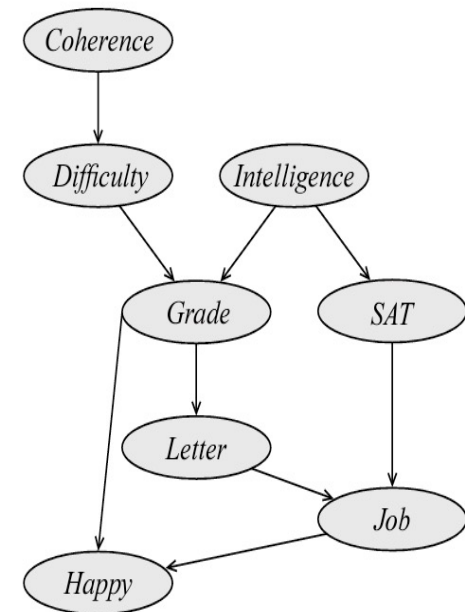
$$\sum_{L,S,G,H} \phi_J(J, L, S) \phi_L(L, G) \phi_H(H, G, J) \psi_3(S, G)$$

- Eliminate H

$$\sum_{L,S,G} \phi_J(J, L, S) \phi_L(L, G) \psi_3(S, G) \sum_H \phi_H(H, G, J)$$

$$\psi_4(G, J) = \sum_H \phi_H(H, G, J)$$

$$\sum_{L,S,G} \phi_J(J, L, S) \phi_L(L, G) \tau_3(S, G) \psi_4(G, J)$$



Variable Elimination

- $P(J)$, order C,D,I,H,G,S,L

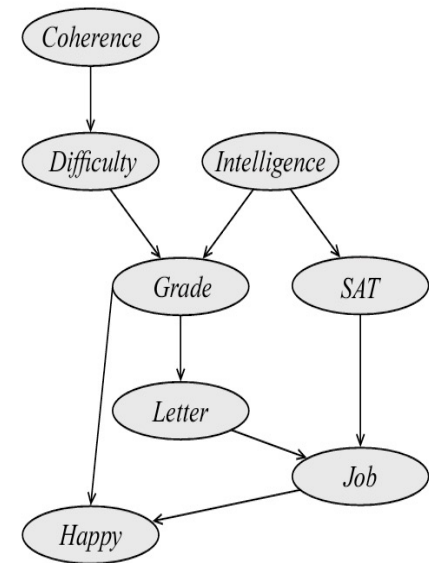
$$\sum_{L,S,G} \phi_J(J, L, S) \phi_L(L, G) \tau_3(S, G) \psi_4(G, J)$$

- Eliminate G

$$\sum_{L,S} \phi_J(J, L, S) \sum_G \phi_L(L, G) \psi_3(S, G) \psi_4(G, J)$$

$$\psi_5(J, L, S) = \sum_G \phi_L(L, G) \tau_3(S, G) \psi_4(G, J)$$

$$\sum_{L,S} \phi_J(J, L, S) \psi_5(J, L, S)$$



Variable Elimination

- $P(J)$, order C,D,I,H,G,S,L

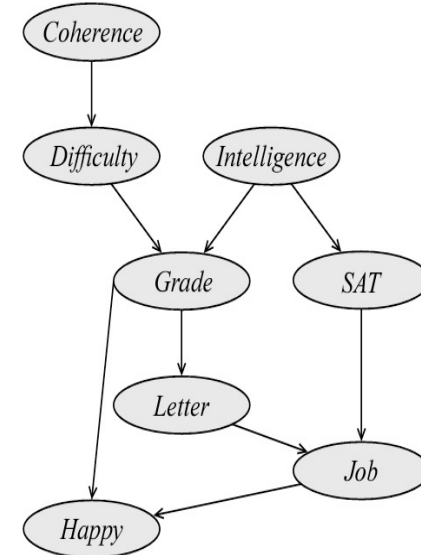
$$\sum_{L,S} \phi_J(J,L,S) \psi_5(J,L,S)$$

- Eliminate S,L

$$\sum_L \sum_S \phi_J(J,L,S) \psi_5(J,L,S)$$

$$\psi_6(J,L) = \sum_S \phi_J(J,L,S) \psi_5(J,L,S)$$

$$\sum_L \psi_6(J,L)$$

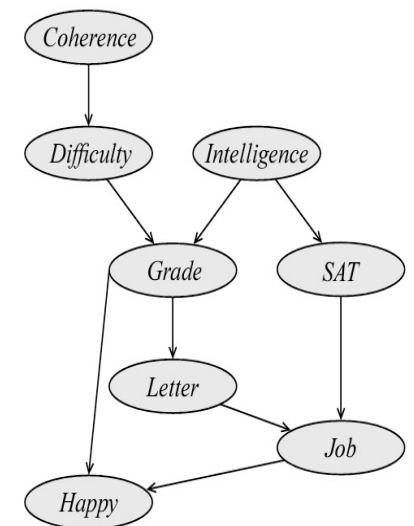


Variable Elimination with evidence

- Goal: $P(J|I = i, H = h) = P(J, I = i, H = h)/P(I = i, H = h)$
- $P(J, I = i, H = h)$
- Eliminate: C, D, G, S, L

$$\sum_{L, S, G, D, C} \phi_J(J, L, S) \phi_L(L, G) \phi_S'(S, I = i) \phi_G'(G, I = i, D) \phi_H'(H = h, G, J) \phi_I'(I = i) \phi_D(C, D) \phi_C(C)$$

Conditional distributions are Gibbs; can be represented as factor graphs!



Variable Elimination for Conditional Probabilities

- Input: Graphical model, set of query variables $\mathbf{Q}$, evidence $\mathbf{E} = \mathbf{e}$
 - Output: factor φ and scalar α
1. Φ = factors in the model
 2. Reduce factors in Φ by $\mathbf{E} = \mathbf{e}$
 3. Choose variable ordering on $\mathbf{Z} = \mathbf{X} - \mathbf{Q} - \mathbf{E}$
 4. $\varphi = \text{Variable-Elimination}(\Phi, \mathbf{Z})$
 5. $\alpha = \sum_{\mathbf{z} \in \text{val}(\mathbf{z})} \varphi(\mathbf{z})$
 6. Return φ, α

Variable Elimination for Bayesian network

- For Bayesian networks, the final factor will be $P(\mathbf{Q}, \mathbf{E} = \mathbf{e})$ and the sum $\alpha = P(\mathbf{E} = \mathbf{e})$.
- This equates to a Gibbs distribution with partition function α .

VE Algorithm Summary

- Algorithm
 - Reduce all factors by evidence
 - Get a set of factors Φ
 - For each non-query variable Z
 - Eliminate-Var Z from Φ
 - Multiply all remaining factors
 - Renormalize to get distribution
- Simple algorithm
- Works for both BN and MN
- Factor product can be done in any order

Complexity of Variable Elimination

- n = number of random variables
- m = number of factors
- In step i , we multiply all factors relating to X_i , resulting in ψ_i , and sum out X_i , giving a new factor τ_i .
 - N_i = number of entries in ψ_i
 - $N_{max} = \max_i N_i$
- If we eliminate everything
 - m initial factors plus n new ones (the τ_i).
 - $m + n$ factors
 - Each is multiplied once, then removed.

Complexity of Variable Elimination

- $m + n$ factors
- Each is multiplied once to produce some ψ_i , then removed to produce τ_i .
 - $(m + n) N_i$ multiplications for X_i
 - $O(mN_{max})$
- Marginalization (summing) touches each entry in each ψ_i once:
 - N_i additions for X_i
 - $O(nN_{max})$

Complexity of Variable Elimination

- Overall $O(mN_{max})$ for n random variables and m factors
 - Bayesian network: $m = n$
 - Markov network: $m \geq n$
- The size N_{max} of the intermediate factors ψ_i is what makes this blow up.
 - v values per random variable
 - k_i variables for factor ψ_i
 - $N_i = v^{k_i}$

Complexity of Variable Elimination

- Complexity of variable elimination linear in
 - size of the model (# factors, # variables)
 - size of the largest factor generated
- Size of factor is exponential in its scope
- Complexity of algorithm depends heavily on elimination ordering

Complexity example

$$\sum_{L,S,G,H,I,D,C} \phi_J(J,L,S) \phi_L(L,G) \phi_S(S,I) \phi_G(G,I,D) \phi_H(H,G,J) \phi_I(I) \phi_D(C,D) \phi_C(C)$$

Start from C

$$\tau_1(D) = \sum_C \phi_C(C) \phi_D(C,D)$$

$$\tau_2(G,I) = \sum_D \phi_G(G,I,D) \tau_1(D)$$

$$\tau_3(S,G) = \sum_I \phi_S(S,I) \phi_I(I) \tau_2(G,I)$$

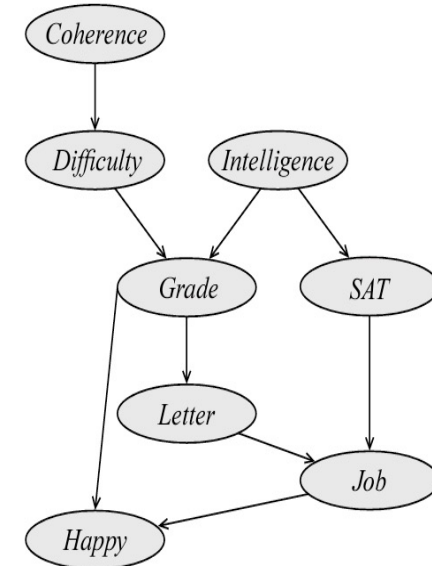
$$\tau_4(G,J) = \sum_H \phi_H(H,G,J)$$

$$\tau_5(L,J) = \sum_G \phi_L(L,G) \tau_3(S,G) \tau_4(G,J)$$

$$\tau_6 = \sum_{L,S} \phi_J(J,L,S) \tau_5(L,J)$$

What if eliminate G first?

$$\tau_1(L,I,D,H,J) = \sum_G \phi_L(L,G) \phi_G(G,I,D) \phi_H(H,G,J)$$



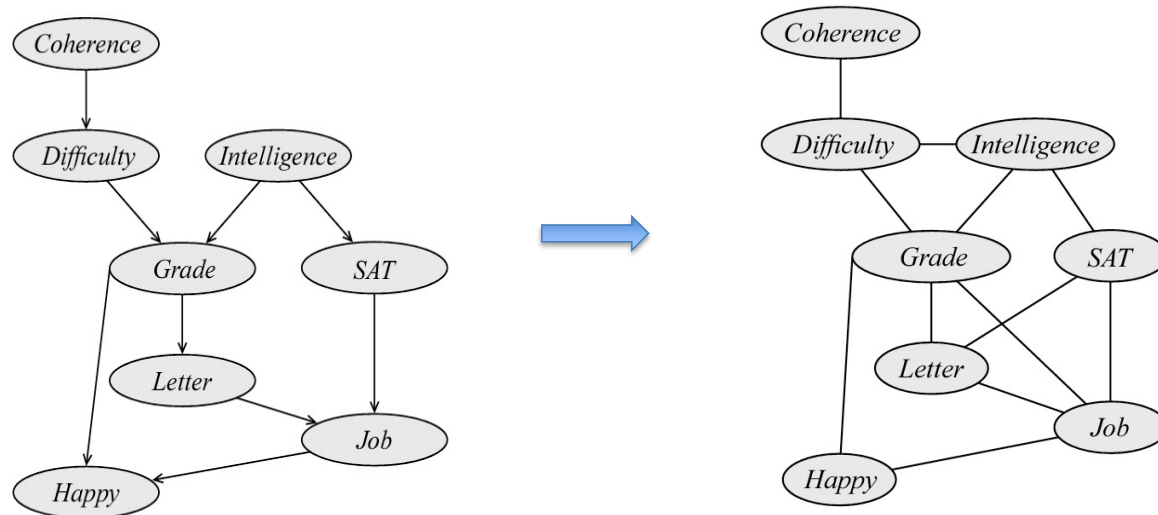
Graph-based perspective

- Can we implement VE on graph?
- VE algorithm operates on undirected graph H_Φ , $\Phi = \{\phi\}$
 - Nodes of $H_\Phi \rightarrow$ variables in $\text{scope}[\Phi]$
- Edge $X_i - X_j$ in $H_\Phi \iff$ there is ϕ in Φ , such that X_i, X_j are in $\text{scope}[\phi]$
 - For BNs, H_Φ should be a moralized graph
 - Each VE step is a transformation on the graph.

Elimination as graph operation

BN $\rightarrow$ MN $\rightarrow$ Moralized graph

$$\phi_J(J, L, S) \phi_L(L, G) \phi_S(S, I) \phi_G(G, I, D) \phi_H(H, G, J) \phi_I(I) \phi_D(C, D) \phi_C(C)$$

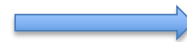
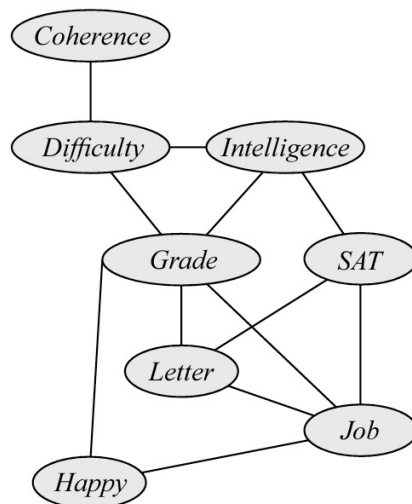


Elimination as graph operation

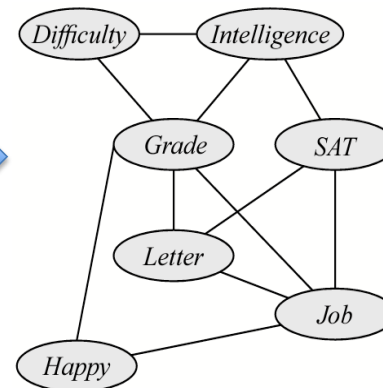
Eliminating C

$$\phi_J(J, L, S) \phi_L(L, G) \phi_S(S, I) \phi_G(G, I, D) \phi_H(H, G, J) \phi_I(I) \phi_D(C, D) \phi_C(C)$$

New factor $\rightarrow \tau_1(D) = \sum_C \underbrace{\phi_D(C, D) \phi_C(C)}_{\text{intermediate factor}}$



Induced MN after eliminating C

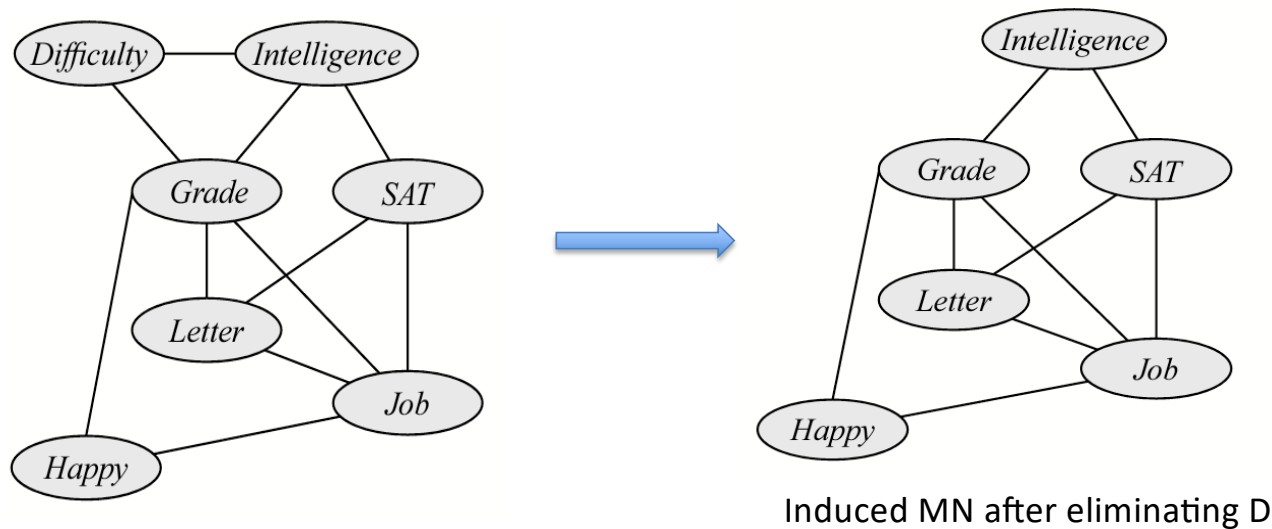


Elimination as graph operation

Eliminating D

$$\tau_2(G, I) = \sum_D \phi_G(G, I, D) \tau_1(D)$$

$$\phi_J(J, L, S) \phi_L(L, G) \phi_S(S, I) \phi_H(H, G, J) \phi_I(I) \tau_2(G, I)$$

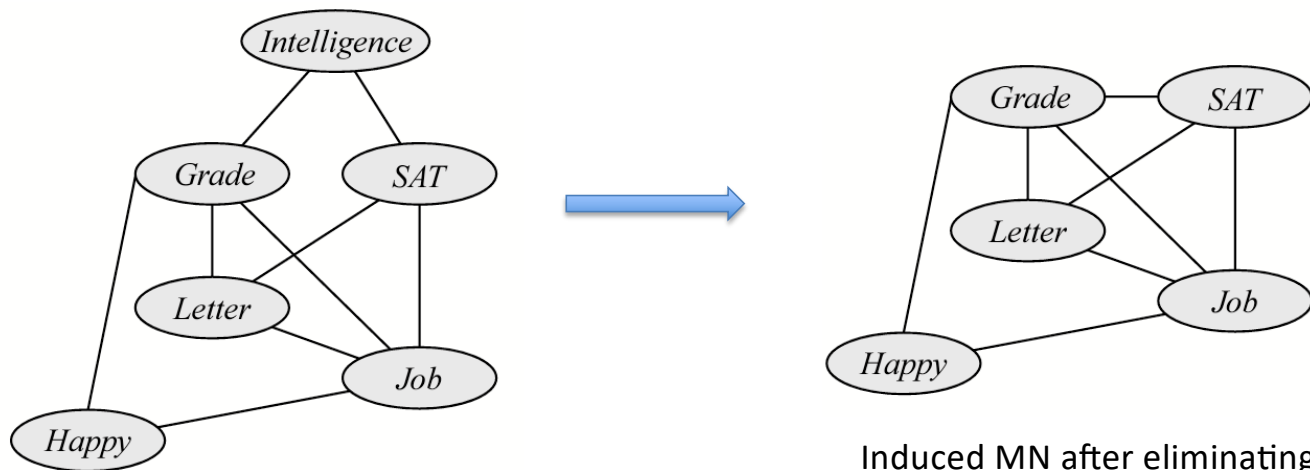


Elimination as graph operation

Eliminating I

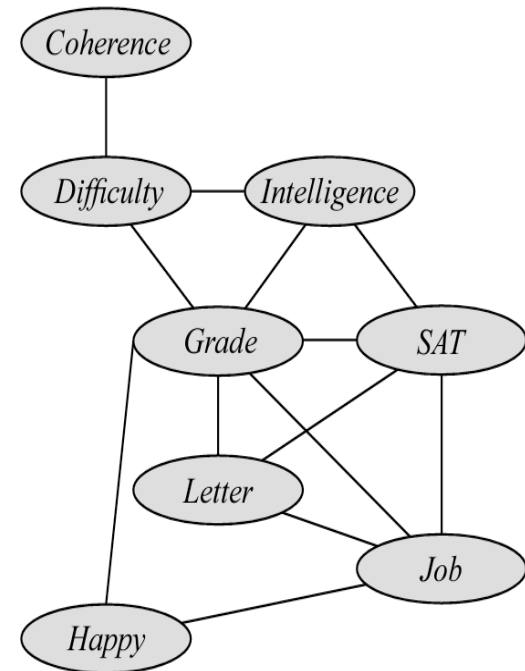
$$\tau_3(S, G) = \sum_I \phi_S(S, I) \phi_I(I) \tau_2(G, I)$$

$$\phi_J(J, L, S) \phi_L(L, G) \phi_H(H, G, J) \tau_3(S, G)$$



Induced Graph

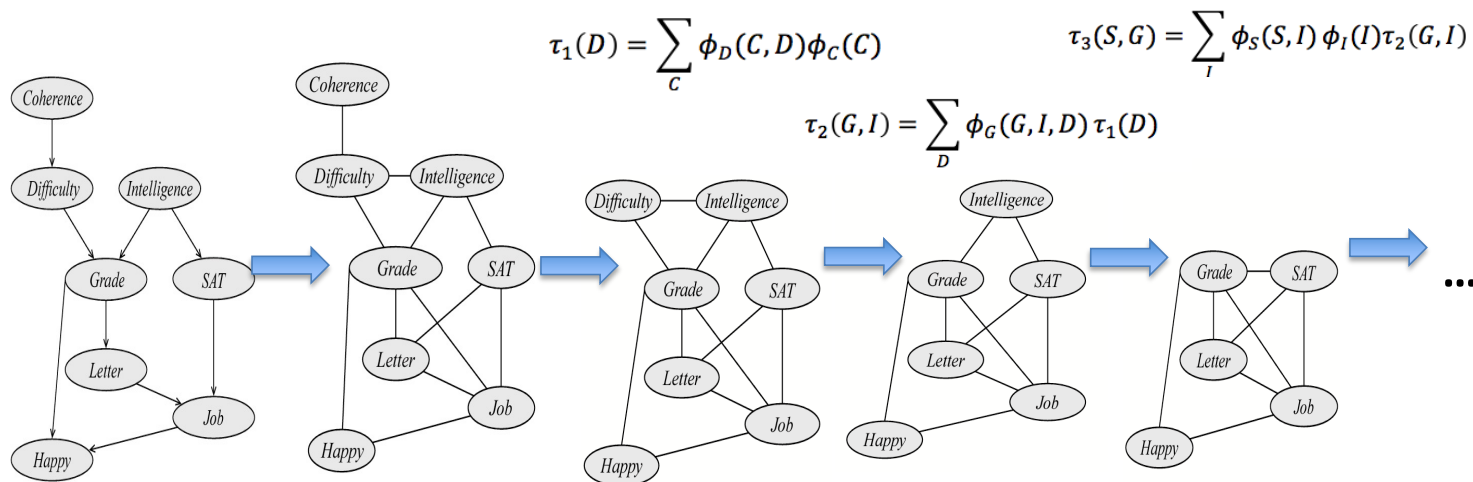
- Take the union over all of the undirected graphs from each step
→ induced graph
 - The scope of every intermediate factor is a clique in this graph.



different ordering → different induced graph

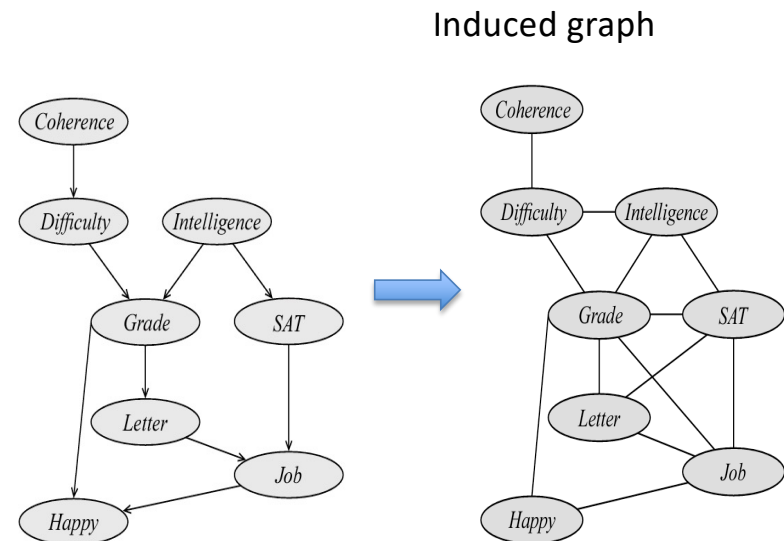
Induced graph

Combining all the graphs



Induced Graph

- The induced graph $I_{\Phi, \alpha}$ over factors Φ and ordering α :
 - Undirected graph
 - X_i and X_j are connected if they appeared in the same factor in a run of the VE algorithm using α as the ordering



How to identify intermediate factors in the VE algorithm from the graph?

Cliques in the Induced Graph

Every maximal clique in the induced graph is the scope of some intermediate factors.

$$\tau_1(D) = \sum_C \phi_C(C) \phi_D(C, D)$$

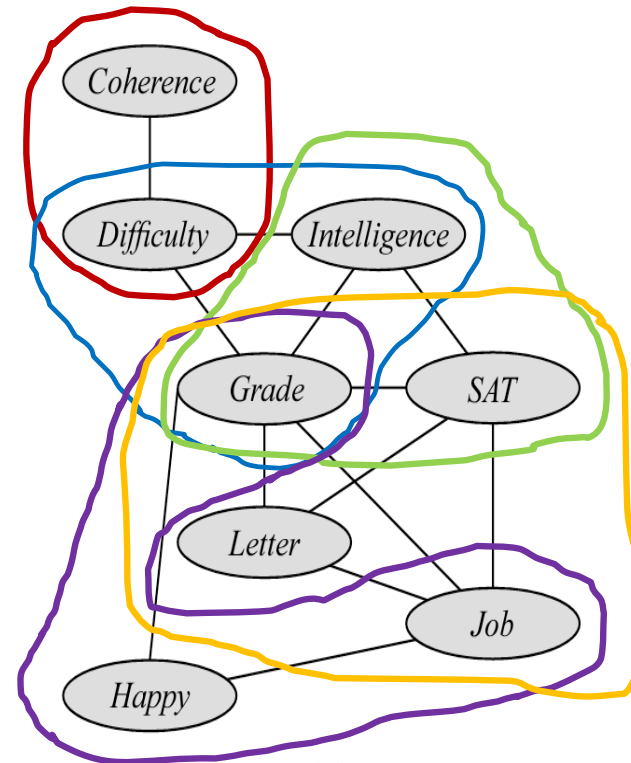
$$\tau_2(G, I) = \sum_D \phi_G(G, I, D) \tau_1(D)$$

$$\tau_3(S, G) = \sum_I \phi_S(S, I) \phi_I(I) \tau_2(G, I)$$

$$\tau_4(G, J) = \sum_H \phi_H(H, G, J)$$

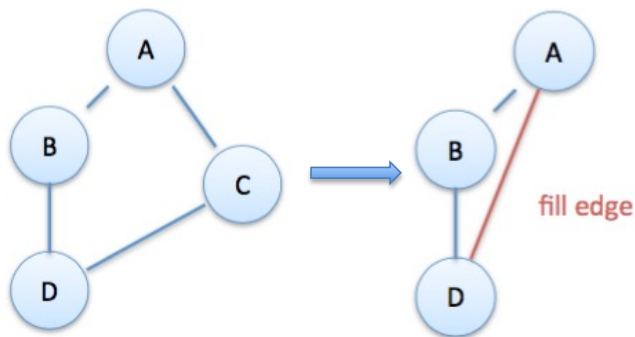
$$\tau_5(L, J) = \sum_G \phi_L(L, G) \tau_3(S, G) \tau_4(G, J)$$

$$\tau_6 = \sum_{L, S} \phi_J(J, L, S) \tau_5(L, J)$$



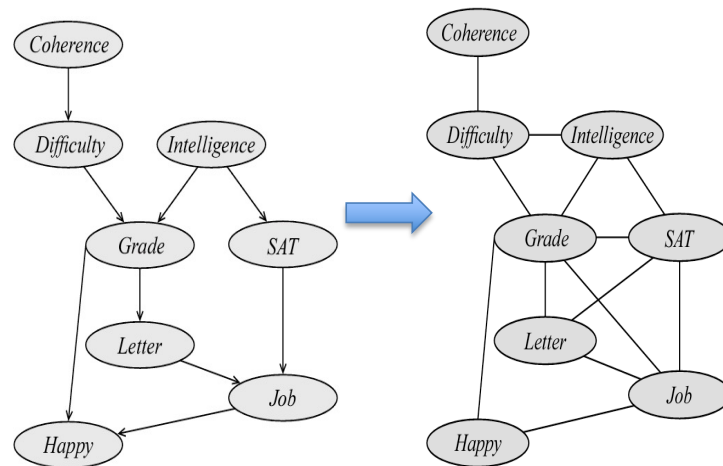
Induced graph and chordal Graphs

- Chordal graph: Undirected graph whose minimal cycles are not longer than 3.
- Induced graphs are always chordal



Eliminating C

Generating a new factor $\rightarrow$ adding fill edge



Elimination as graph operation

- Variable elimination can be viewed as transformations on undirected graph
 - Elimination connects all node's current neighbors
- Cliques in resulting induced graph directly correspond to algorithm's complexity

How can we estimate complexity using graphs?

Definitions: Induced Width and Tree Width

- Induced Width: Number of nodes in the largest clique of the induced graph, minus one.
 - Relative to an ordering
- Tree width: minimum width over all possible orderings.
 - Bound on the best performance we can hope for
- VE runtime is exponential in tree width

How can we find the best ordering?